

Q1 List all the tasks in requirement engineering. Explain it in brief.

Q2 Explain requirement engineering, describe its process, and discuss the challenges in requirement elicitation.

Q8 What is requirement engineering? State its process and explain the requirement elicitation problem.

OR Explain the task done during requirement engineering. What is requirement elicitation? Explain why it is considered as a difficult task.

Ans Requirement Engineering is the process that:

- Begins during the **communication activity**
- Continues into the **modeling activity**
- Builds a bridge from system requirements to software design and construction

It helps understand:

- The context of the software
- User needs
- System priorities
- Functional behavior

## Tasks in Requirement Engineering

Requirement Engineering consists of **seven distinct tasks** that help define what the customer wants and establish a strong foundation for software design and development

### 1. Inception

- Begins the requirement gathering process
- Identifies stakeholders
- Establishes communication
- Recognizes different viewpoints

This task helps initiate collaboration and understand the problem.

### 2. Elicitation

- Focuses on gathering requirements from stakeholders
- Identifies system boundaries and objectives
- May involve collaborative meetings and requirement analysis

Helps determine what the system should do.

### 3. Elaboration

- Refines information obtained during inception and elicitation
- Develops technical models of system functions and features
- Includes use cases and domain classes

This task focuses on analysis modeling.

#### **4. Negotiation**

- Resolves conflicts between customer needs and project constraints
- Prioritizes requirements
- Evaluates risks and development effort

Ensures balanced decision-making.

#### **5. Specification**

- Documents the requirements
- Can be in the form of:
  - Written documents
  - Models
  - Use cases
  - Prototypes

Provides a clear description of system needs

#### **6. Validation**

- Ensures requirements are correct and complete
- Identifies:
  - Missing information
  - Conflicts
  - Inconsistencies

Helps improve requirement accuracy.

#### **7. Requirements Management**

- Maintains requirements throughout the project
- Assigns unique identifiers
- Tracks dependencies and changes

Ensures proper handling of evolving requirements.

### **Challenges in Requirement Elicitation**

Requirement elicitation is difficult due to the following challenges:

#### **1. Problems of Scope**

- Difficulty in identifying system boundaries
- Risk of specifying too much technical detail

#### **2. Problems of Understanding**

- Stakeholders may not clearly know what they need
- Important information may be assumed as obvious and omitted

### 3. Problems of Volatility

- Requirements change over time
- Makes requirement gathering unstable

**Q3 Distinguish between functional and nonfunctional requirements.**

Ans

| Parameter                 | Functional Requirements                                                                         | Non-Functional Requirements                                                                          |
|---------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Meaning</b>            | Functional requirements describe what the system should do in terms of operations and services. | Non-functional requirements describe how the system should perform while carrying out its functions. |
| <b>Focus</b>              | Functional requirements focus on the functionality of the system.                               | Non-functional requirements focus on the quality of the system.                                      |
| <b>Nature</b>             | Functional requirements define the operations that the system must perform.                     | Non-functional requirements define the performance and constraints of the system.                    |
| <b>System Description</b> | Functional requirements specify the behavior of the system.                                     | Non-functional requirements specify the characteristics of the system.                               |
| <b>User Interaction</b>   | Functional requirements are directly related to user tasks and interactions.                    | Non-functional requirements are indirectly related to user experience.                               |
| <b>Examples</b>           | Functional requirements include authentication, reporting and data processing.                  | Non-functional requirements include performance, security and availability.                          |
| <b>Objective</b>          | The objective of functional requirements is to define system functions.                         | The objective of non-functional requirements is to define system efficiency and reliability.         |
| <b>Role</b>               | Functional requirements describe what services the system provides.                             | Non-functional requirements describe the quality of the services provided by the system.             |

- Functional requirements include system actions such as:
  - Authentication
  - Evaluation management
  - Submission engine
  - Reporting
- Non-functional requirements define system qualities such as:
  - Performance
  - Security

- Availability
- Usability

#### Q4 Design use case diagram for user interaction with ATM system.

Ans A **Use Case** describes the interaction between an actor and the system to achieve a specific goal

Actors are entities that interact with the system and are external to it

#### Actors in ATM System

- Customer (Primary Actor)
- Bank System (Secondary Actor)

Actors initiate interaction and use the system to fulfill goals

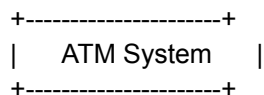
#### Use Cases

The ATM system supports the following user interactions:

- Insert Card
- Enter PIN
- Withdraw Cash
- Deposit Cash
- Check Balance
- Transfer Funds
- Print Receipt

Each use case represents a sequence of actions performed by the user to achieve a specific task

#### Use Case Diagram (Textual Representation)



Customer ----> Insert Card  
 Customer ----> Enter PIN  
 Customer ----> Withdraw Cash  
 Customer ----> Deposit Cash  
 Customer ----> Check Balance  
 Customer ----> Transfer Funds  
 Customer ----> Print Receipt

Bank System ----> Validate PIN  
 Bank System ----> Process Transaction

#### Explanation

- The **Customer** interacts directly with the ATM to perform transactions.

- The **Bank System** supports backend operations such as authentication and transaction processing.

This diagram illustrates how users interact with the ATM system through various use cases

**Q5 Design a Use Case Diagram for a University Enrollment System, illustrating the interactions between actors (such as Students, Administrators, and Professors) and various use cases like course registration, payment processing, and student record management.**

Ans

A **Use Case Diagram** describes how external actors interact with a system to achieve specific goals

Actors are people or systems that communicate with the software and are external to it.

### Actors in University Enrollment System

According to stakeholder concepts

- Student
- Administrator
- Professor

Each actor interacts with the system to perform specific tasks.

### Use Cases

The University Enrollment System includes the following interactions:

- Course Registration
- Payment Processing
- Student Record Management
- View Course Details
- Update Student Records
- Manage Enrollment

Each use case represents a functional interaction between the actor and the system

### Use Case Diagram (Textual Representation)

```

+-----+
| University Enrollment System |
+-----+

```

Student -----> Course Registration

Student -----> Payment Processing

Student -----> View Course Details

Administrator --> Manage Enrollment

Administrator --> Student Record Management

Professor -----> Update Student Records

Professor -----> View Course Details

### Explanation

- **Students** register for courses and complete payments
- **Administrators** manage enrollment and student records
- **Professors** update academic information and view course details

This diagram illustrates how different actors interact with the University Enrollment System to perform essential enrollment-related tasks

**Q6 Explain four desirable characteristics of a good software requirement specification document.**

Ans A good Software Requirement Specification (SRS) should clearly define system needs and support validation of requirements. The following are four important characteristics:

#### 1. Correctness

- Each requirement should align with overall system objectives
- The SRS must accurately represent user needs

This ensures that the system solves the intended problem.

#### 2. Completeness

- The document should not have missing information
- All required functionalities and constraints must be included

Helps avoid confusion during development.

#### 3. Consistency

- Requirements should not conflict with each other
- The document must present a unified view of system behavior

Prevents errors during implementation.

#### 4. Unambiguity

- Each requirement must be clearly defined
- Avoid vague or unclear statements

Ensures all stakeholders interpret requirements in the same way

### Q7 Distinguish requirement negotiation and validation? States its technique purpose of Requirements Validation

Ans Requirement Engineering includes both negotiation and validation to ensure that requirements are feasible and correct

| Parameter       | Requirement Negotiation                                              | Requirement Validation                        |
|-----------------|----------------------------------------------------------------------|-----------------------------------------------|
| <b>Purpose</b>  | Resolves conflicts between stakeholder needs and project constraints | Ensures requirements are correct and complete |
| <b>Focus</b>    | Balances functionality with cost and time                            | Detects errors and inconsistencies            |
| <b>Activity</b> | Prioritizes and modifies requirements                                | Reviews requirements for correctness          |
| <b>Outcome</b>  | Achieves agreement among stakeholders                                | Produces validated requirement model          |

### Techniques of Requirement Negotiation

Requirement negotiation involves:

- Identifying stakeholders
- Establishing their win conditions
- Reconciling them into win-win conditions

It also includes:

- Ranking requirements
- Analyzing risks
- Estimating development effort

### Purpose of Requirement Validation

Requirement validation is a review mechanism used to identify:

- Errors in content
- Missing information
- Inconsistencies
- Conflicting or unrealistic requirements

It ensures that:

- Each requirement aligns with system objectives
- Requirements are well defined
- Requirements are achievable and testable

**Q9 Apply requirement engineering activities to identify both functional and non-functional requirements for a College ERP system.**

Ans Requirement Engineering consists of activities that help identify system needs and establish both functional and non-functional requirements

### **1. Inception**

In this stage, stakeholders are identified such as:

- Students
- Faculty
- Administrators

Each stakeholder provides a different viewpoint about the system.

### **2. Elicitation**

Requirements are gathered through interaction with stakeholders.

- Stakeholders express their needs regarding system usage
- System scope and objectives are understood

### **3. Elaboration**

System functions are refined by developing models such as:

- Student enrollment
- Fee payment
- Result processing

This helps in understanding system behavior.

### **4. Negotiation**

- Important features are prioritized
- Balance is maintained between functionality, cost and time

### **5. Specification**

In this stage, Functional and Non-Functional requirements are formally defined.

#### **Functional Requirements**

The College ERP system should support:

- Student Registration
- Course Management
- Attendance Tracking
- Examination Management



- Report Generation

These describe system operations.

## Non-Functional Requirements

The system should ensure:

- **Performance:** The system should process requests efficiently and support multiple users.
- **Security:** The system should protect sensitive academic and administrative data.
- **Availability:** The system should remain operational whenever required.
- **Usability:** The system should be easy for students and staff to use.

These define system quality attributes.

## 6. Validation

Requirements are reviewed to ensure they are:

- Complete
- Consistent
- Achievable

## 7. Requirements Management

- Requirements are assigned identifiers
- Changes are tracked throughout development

### Q10 Explain Software Requirement Specification.

Ans A **Software Requirement Specification (SRS)** is a formal document that clearly describes what the software system should do and how it should perform. It includes System functions, Interfaces, Constraints and gives a complete description of system requirements.

SRS serves the following purposes

- 1. Specifies Software Characteristics :** Defines how the software should operate->Indicates how it interacts with other systems
- 2. Establishes Constraints :** Identifies limitations the software must follow
- 3. Provides a Basis for Design:** Helps designers convert requirements into system architecture
- 4. Supports Quality Assessment:** Enables both developers and customers to assess software after development

### Objectives of SRS

- 1. To Describe Customer Requirements :** Clearly defines what the customer expects from the system

**2. To Establish a Basis for Software Design:**Serves as input for system design

**3. To Define Validatable Requirements:**Provides requirements that can be checked once the software is built

### **Role of SRS**

SRS acts as a bridge between:Requirement analysis and Software design

It ensures that:

- Developers understand system needs
- Designers create accurate solutions
- Customers can verify whether the system meets expectations

**Q11 Explain requirement Elicitation Process.**

**Q12 List down requirement Elicitation Techniques**

Ans Requirement Elicitation is the process of gathering system requirements from stakeholders.

It is considered one of the most important tasks in Requirement Engineering because it helps define what the system should do

### **Activities/techniques in Requirement Elicitation**

Elicitation is carried out through the following activities:

#### **1. Collaborative Requirements Gathering**

- Meetings are conducted with:
  - Software engineers
  - Customers
  - Stakeholders

Guidelines include:

- Establish rules for participation
- Use an agenda to guide discussion
- Appoint a facilitator
- Use tools such as worksheets, charts or electronic forums

The goal is to:

- Identify the problem
- Propose solutions
- Negotiate different approaches
- Define preliminary requirements

#### **2. Quality Function Deployment (QFD)**

This technique translates customer needs into technical requirements

It identifies three types of requirements:

- **Normal Requirements** – explicitly stated by customers
- **Expected Requirements** – implicit but necessary
- **Exciting Requirements** – features beyond expectations

### 3. Usage Scenarios

- Engineers and users collaborate to define system usage
- Helps understand how system functions will be used

### Elicitation Work Products

The process produces:

- Statement of need and feasibility
- Defined system scope
- List of stakeholders
- Description of system environment
- List of functional requirements
- Preliminary usage scenarios

### Q13 Illustrate the components of an SRS for a computerized library system

Ans A Software Requirement Specification (SRS) defines system requirements including purpose, users, functions and constraints. For a **Computerized Library System**, the SRS includes the following components:

#### 1. Project Purpose & Scope

- Defines the problem to be solved
- Specifies system boundaries
- Example:
  - Replacing manual library records with a digital system

#### 2. Stakeholders (User Persons)

Identifies system users such as:

- Librarian
- Students
- Faculty
- System Administrator

#### 3. Functional Requirements

Defines system operations such as:

- Book search
- Issue and return of books
- User registration
- Record management

#### **4. Non-Functional Requirements**

Specifies system qualities like:

- Performance
- Security
- Availability
- Usability

#### **5. System Architecture & Data Flow**

- Shows how data moves in the system
- Defines interaction between modules

#### **6. Key Use Cases**

Examples:

- User searches for book
- Librarian issues book
- System generates reports

#### **7. Constraints & Assumptions**

- Defines system limitations
- Example:
  - Works within institutional network

#### **8. Project Timeline**

- Requirement analysis
- Design
- Development
- Testing & Deployment

#### **Q14 List down Software Requirements Characteristics.**

Ans A good software requirement should possess certain important characteristics to ensure clarity and correctness. The key characteristics are:

##### **1. Aligned with Objectives**

- Each requirement should match overall system or product goal

##### **2. Appropriate Level of Abstraction**

- Requirements should not include unnecessary technical details

### **3. Necessary**

- Each requirement should be essential to the system
- Optional features should be clearly identified

### **4. Well-defined and Unambiguous**

- Requirements should be clearly stated
- Avoid vague or unclear descriptions

### **5. Attributed**

- Each requirement should have a clear source or stakeholder

### **6. Non-Conflicting**

- Requirements should not contradict one another

### **7. Attainable**

- Requirements should be achievable within the technical environment

### **8. Testable**

- Each requirement should be verifiable after implementation

### **Q15 Explain the importance of Requirement engineering.**

Ans Requirement Engineering is important because it establishes a strong foundation for the software system to be developed. It helps bridge the gap between system needs and software design.

### **Key Importance**

#### **1. Provides Clear Understanding of System Needs**

Requirement Engineering helps examine:

- The context of software work
- The specific needs that design and construction must address

#### **2. Helps Set Development Priorities**

It identifies: The priorities that guide the order in which work should be completed

#### **3. Defines System Information and Behavior**

Requirement Engineering clarifies:

- The information
- Functions
- Behavior

that influence system design

#### 4. Establishes a Solid Foundation

It ensures that the system to be developed matches user needs.

Without proper requirements, software projects may fail

#### 5. Supports Design and Development

Requirement Engineering:

- Builds a bridge from requirements to software construction

**Q16 Model the workflow of a Train Reservation System using an Activity Diagram.**

Ans An **Activity Diagram** represents the flow of interaction within a specific scenario using flowchart-like symbols. It shows how actions are performed step-by-step.



#### Explanation

- Rounded rectangles represent actions such as entering travel details or making payment
- Arrows show flow of control
- Decision points indicate seat availability

The workflow of Train Reservation System includes:

1. The process starts when the user enters travel details.
2. The system searches for available trains.

3. The system checks whether seats are available.
4. If seats are not available, the system displays a message.
5. If seats are available, the user enters passenger details.
6. The user proceeds to make payment.
7. The system confirms the booking.
8. Finally, the ticket is generated.

This activity diagram illustrates the step-by-step process of reserving a train ticket from search to confirmation.

**Q 17 Explain about various categories of non functional requirements and their importance .**

Ans Non-functional requirements describe the quality attributes of a software system such as performance, security and usability. They define how well the system performs rather than what it does.

### **1. Performance Requirements**

- Specify how efficiently the system operates
- Example:
  - Handling large number of users with low response time

### **2. Security Requirements**

- Define protection mechanisms
- Example:
  - Encryption of stored data
  - Secure access control

### **3. Availability Requirements**

- Ensure system is operational when needed
- Example:
  - High uptime during critical usage periods

### **4. Usability Requirements**

- Define ease of use for users
- Example:
  - Accessibility standards for users

### **Importance of Non-Functional Requirements**

**1. Improve System Quality** -Ensure system performance and reliability

**2. Enhance User Experience**- Support usability and accessibility

**3. Ensure System Reliability** -Maintain availability during usage

**4. Provide Security**- Protect system data and operations

**Q18 Explain the phases of the unified process and how it supports incremental and iterative development.**

**Ans Phases of the Unified Process**

The Unified Process follows an iterative and incremental approach and consists of four major phases

### **1. Inception**

- Defines project scope and objectives
- Identifies system requirements
- Establishes overall goals

This phase initiates the development process.

### **2. Elaboration**

- Refines system requirements
- Develops system models such as:
  - Use cases
  - Class diagrams

Helps understand system functionality and behavior.

### **3. Construction**

- Involves actual development of the system
- Builds software components
- Produces functional software

### **4. Transition**

- Delivers the system to users
- Includes deployment and feedback

## **Support for Incremental and Iterative Development**

The Unified Process supports development by:

### **1. Iterative Development**

- The system evolves through repeated cycles
- Each phase is revisited and refined over time

### **2. Incremental Delivery**

- Each cycle produces an improved version of the system



- Software functionality is gradually expanded

**Q19 Explain in brief the collaborative requirement gathering activity.**

**Ans Collaborative Requirement Gathering**

Collaborative Requirement Gathering is an activity used during requirement elicitation to collect system requirements through interaction between stakeholders and developers

It involves structured meetings to identify system needs.

## **Key Features**

### **1. Joint Participation**

Meetings are conducted with:

- Software engineers
- Customers
- Other stakeholders

This ensures multiple viewpoints are considered.

### **2. Defined Rules**

- Rules for preparation and participation are established
- Helps maintain effective communication

### **3. Structured Agenda**

- Meetings follow an agenda
- Ensures all important topics are covered while allowing open discussion

### **4. Facilitator**

- A facilitator manages the discussion
- Encourages productive interaction

### **5. Use of Supporting Tools**

- Tools such as worksheets, charts or electronic forums are used
- Helps record ideas and requirements

## **Objective**

The main goal is to:

- Identify problems
- Propose solutions
- Negotiate approaches
- Define preliminary requirements

**Q20 Explain various functional and nonfunctional requirements for the Bank ATM System using requirements engineering processes**

**Ans 1. Inception**

Identify stakeholders such as:

- Bank customers
- Bank administrators

Different stakeholders provide system expectations.

**2. Elicitation**

Requirements are gathered through interaction with stakeholders.

- System objectives are understood
- User needs regarding ATM services are collected

**3. Elaboration**

Develop models and use cases such as:

- Withdraw Cash
- Check Balance
- Transfer Funds

This helps refine system functionality.

**4. Negotiation**

- Prioritize important ATM features
- Balance system capabilities with cost and time constraints

**5. Specification**

In this stage, Functional and Non-Functional requirements are formally defined.

**Functional Requirements**

The ATM system should support:

- User authentication through PIN validation
- Cash withdrawal
- Balance inquiry
- Fund transfer
- Receipt generation

These describe system operations.

**Non-Functional Requirements**

The ATM system should ensure:

- **Performance:** The system should process transactions quickly.
- **Security:** The system should protect user data and prevent unauthorized access.
- **Availability:** The system should remain operational at all times.
- **Usability:** The system should be easy for customers to operate.

These define system quality attributes.

## 6. Validation

Ensure requirements are:

- Consistent
- Feasible
- Testable

## 7. Requirements Management

- Assign unique identifiers to requirements
- Track changes during development